Homework 5

6.990.)

a.

We can solve for the model parameters by reformatting the problem as a least squares problem. Define matrix A, input x, and output y to be

$$A = \begin{bmatrix} x_1^{(1)}(1) & x_2^{(1)}(1) & ... & x_n^{(1)}(1) & 1 & 0 & 0 & ... & 0 \\ x_1^{(2)}(1) & x_2^{(2)}(1) & ... & x_n^{(2)}(1) & 0 & 1 & 0 & ... & 0 \\ x_1^{(3)}(1) & x_2^{(3)}(1) & ... & x_n^{(3)}(1) & 0 & 0 & 1 & ... & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ x_1^{(K)}(1) & x_2^{(K)}(1) & ... & x_n^{(K)}(1) & 0 & 0 & 0 & ... & 1 \\ x_1^{(1)}(2) & x_2^{(1)}(2) & ... & x_n^{(1)}(2) & 1 & 0 & 0 & ... & 0 \\ x_1^{(2)}(2) & x_2^{(2)}(2) & ... & x_n^{(2)}(2) & 0 & 1 & 0 & ... & 0 \\ x_1^{(3)}(2) & x_2^{(3)}(2) & ... & x_n^{(3)}(2) & 0 & 0 & 1 & ... & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ x_1^{(K)}(2) & x_2^{(K)}(2) & ... & x_n^{(K)}(2) & 0 & 0 & 0 & ... & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x_1^{(1)}(T) & x_2^{(1)}(T) & ... & x_n^{(1)}(T) & 1 & 0 & 0 & ... & 0 \\ x_1^{(2)}(T) & x_2^{(2)}(T) & ... & x_n^{(2)}(T) & 0 & 1 & 0 & ... & 0 \\ x_1^{(3)}(T) & x_2^{(3)}(T) & ... & x_n^{(3)}(T) & 0 & 0 & 1 & ... & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ x_1^{(K)}(T) & x_2^{(K)}(T) & ... & x_n^{(K)}(T) & 0 & 0 & 0 & ... & 1 \end{bmatrix} \in \mathbb{R}^{(KT)\times(n+K)}$$

$$x = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \\ b^{(1)} \\ b^{(2)} \\ \vdots \\ b^{(k)} \end{bmatrix} \in \mathbb{R}^{n+K}$$

$$y = \begin{bmatrix} y^{(1)}(1) \\ y^{(2)}(1) \\ \vdots \\ y^{(K)}(1) \\ y^{(1)}(2) \\ y^{(2)}(2) \\ \vdots \\ y^{(K)}(2) \\ \vdots \\ y^{(1)}(T) \\ y^{(2)}(T) \\ \vdots \\ y^{(K)}(T) \end{bmatrix} \in \mathbb{R}^{KT}$$

Then we have

$$\|Ax-y\|^2 = \| \begin{bmatrix} a^\intercal x^{(1)}(1)+b^{(1)} \\ a^\intercal x^{(2)}(1)+b^{(2)} \\ \vdots \\ a^\intercal x^{(K)}(1)+b^{(K)} \\ a^\intercal x^{(1)}(2)+b^{(1)} \\ a^\intercal x^{(2)}(2)+b^{(2)} \\ \vdots \\ a^\intercal x^{(K)}(2)+b^{(K)} \\ \vdots \\ a^\intercal x^{(1)}(T)+b^{(1)} \\ a^\intercal x^{(2)}(T)+b^{(2)} \\ \vdots \\ a^\intercal x^{(K)}(T)+b^{(K)} \end{bmatrix} - \begin{bmatrix} y^{(1)}(1) \\ y^{(2)}(1) \\ \vdots \\ y^{(K)}(1) \\ y^{(1)}(2) \\ y^{(2)}(2) \\ \vdots \\ y^{(K)}(2) \\ \vdots \\ y^{(1)}(T) \\ y^{(2)}(T) \\ \vdots \\ y^{(K)}(T) \end{bmatrix} \|^2 = \sum_{t=1}^{T}\sum_{k=1}^{K}(a^\intercal x^{(k)}(t)+b^{(k)}-y^{(k)}(t))^2 = (TK)E$$

As T and K are constants, the x that minimizes $\|Ax-y\|^2$ will minimize E as well. Since x is a vector of the model parameters, that means x_{ls} gives the model parameters which minimize the mean square error. Hence, the optimial model parameters are defined to be

$$\begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \\ b^{(1)} \\ b^{(2)} \\ \vdots \\ b^{(k)} \end{bmatrix} = x_{ls} = (A^\intercal A)^{-1}A^\intercal y$$

*Note for this solution to hold, A must be full rank and skinny, which depends on $KT \geq n+K$ and the columns of A being independent of each other (depends on x data).

b.

This question was done in Julia, the code and results are shown on the next page. As can be seen, the mean square error with distinct offsets is 0.1098 and the mean square error with a common offset is 7.267. This means the model with distinct offsets is significantly more accurate than the model with a common offset. Also, when looking at the distinct offsets $b^{(1)},...,b^{(k)}$, we observe that $b^{(7)}$ is noticeable because it has a value of 9.538 whereas every other offset has a value below 1. This difference indicates the maintenance crew should check out vehicle 7.

```julia
using LinearAlgebra
using Statistics

#Question 1.)

#Reading Data
include("readclassjson.jl")
data = readclassjson("fleet_mod_data.json")

n = data["n"]
K = data["K"]
T = data["T"]
x_array = [data["X$i"]' for i=1:data["K"]]
y_array = [data["y$i"]' for i=1:data["K"]]


#Creating y vector and x portion of A matrix
x_matrix = vcat(x_array...)
y = vcat(y_array...)

#Creating identity portion of A matrix and combining
b_matrix = zeros(T*K, K)
for k in 1:K
    rows = (T*(k-1)+1):(T*k)
    b_matrix[rows, k] .= 1.0
end
A = [x_matrix b_matrix]

#Solving Least Squares
x_ls = A \ y
a = x_ls[1:n]
b = x_ls[n+1:end]

println()
println("Model parameter a = ", a)
println("Model parameter [b(1), ..., b(k)] = ", b)
println("Mean Square Error E = ", norm((A*x_ls) - y)^2 / (T*K))
println()

A_com = [x_matrix ones(T*K)]
x_com = A_com \ y
a_com = x_com[1:n]
b_com = x_com[n+1:end]
println("Common Offset Model parameter a = ", a_com)
println("Common Offset Model parameter b = ", b_com)
println("Common Offset Mean Square Error Ecom = ", norm((A_com*x_com) - y)^2 / (T*K))
```

```
Model parameter a = [0.3251000862962055, -0.06178583337985051, 0.4452993012351774, -0.2147648252017308, -0.7930617650043401]
Model parameter [b(1), ..., b(k)] = [0.879930939223189, 0.7308918997269852, 0.30566627875106284, 0.2562491671748455, 0.3840904202897004, 0.899
3779192762728, 9.537545707104035, 0.9006659587672774, 0.5403818743777363, 0.8925064038814047]
Mean Square Error E = 0.10975023243532886

Common Offset Model parameter a = [0.36113292468143254, -0.18276699715037859, 0.46603063317001225, -0.15964151030248572, -0.827410898455250
4]
Common Offset Model parameter b = [1.5245652971807755]
Common Offset Mean Square Error Ecom = 7.2674841563113555
```

6.2300.)

a.

For $k = 5$ and $n = 10$,

$$C = \begin{bmatrix} 1 & 0 & 0 & ... & 0 \\ 1 & 0 & 0 & ... & 0 \\ 1 & 0 & 0 & ... & 0 \\ 1 & 0 & 0 & ... & 0 \\ 1 & 0 & 0 & ... & 0 \\ 0 & 1 & 0 & ... & 0 \\ 0 & 1 & 0 & ... & 0 \\ 0 & 1 & 0 & ... & 0 \\ 0 & 1 & 0 & ... & 0 \\ 0 & 1 & 0 & ... & 0 \\ 0 & 0 & 1 & ... & 0 \\ 0 & 0 & 1 & ... & 0 \\ 0 & 0 & 1 & ... & 0 \\ 0 & 0 & 1 & ... & 0 \\ 0 & 0 & 1 & ... & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & ... & 1 \\ 0 & 0 & 0 & ... & 1 \\ 0 & 0 & 0 & ... & 1 \\ 0 & 0 & 0 & ... & 1 \\ 0 & 0 & 0 & ... & 1 \end{bmatrix} \in \mathbb{R}^{kn \times n} = \mathbb{R}^{50 \times 10}$$

We can generalize this matrix for arbitrary n and k. Start with a zero matrix of size $kn \times n$. Replace the first k rows of the first column with 1. Then replace the second k rows (rows $k+1$ to $2k$) of the second column with 1. Keep on going until you replace the nth k rows of the nth column with 1.

b.

Start with some specific examples for this problem.

$$y_1 = w_1 + \sum_{j=1}^{11} r_{1-j} u_j = w_1 + r_0 u_1 + r_{-1} u_2 + ... + r_{-10} u_{11}$$

$$y_2 = w_2 + \sum_{j=1}^{12} r_{2-j} u_j = w_2 + r_1 u_1 + r_0 u_2 + ... + r_{-10} u_{12}$$

$$y_{50} = w_{50} + \sum_{j=40}^{50} r_{50-j} u_j = w_{50} + r_{10} u_{40} + r_9 u_{41} + ... + r_0 u_{50}$$

We can conclude that for $m = 50$ and $q = 10$,

$$B = \begin{bmatrix} r_0 & r_{-1} & r_{-2} & ... & r_{-10} & 0 & 0 & ... & 0 & 0 \\ r_1 & r_0 & r_{-1} & ... & r_{-9} & r_{-10} & 0 & ... & 0 & 0 \\ r_2 & r_1 & r_0 & ... & r_{-8} & r_{-9} & r_{-10} & ... & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ r_{10} & r_9 & r_8 & ... & r_0 & r_{-1} & r_{-2} & ... & 0 & 0 \\ 0 & r_{10} & r_9 & ... & r_1 & r_0 & r_{-2} & ... & 0 & 0 \\ 0 & 0 & r_{10} & ... & r_2 & r_1 & r_0 & ... & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & ... & 0 & 0 & 0 & ... & r_0 & r_{-1} \\ 0 & 0 & 0 & ... & 0 & 0 & 0 & ... & r_1 & r_0 \end{bmatrix} \in \mathbb{R}^{m \times m} = \mathbb{R}^{50 \times 50}$$

Similar to C, this matrix can also be generalized for arbitrary m and q. Start with a zero matrix of size $m \times m$. Put r_0 along the top left to bottom right diagonal. To the right of each r_0, put an r_{-1}, then put an r_{-2} to the right of each r_{-1} and keep going until you place the r_{-q}. When you hit the right boundary of the matrix, you stop for that row. Similarly, to the left of each r_0, put an r_1, then put an r_2 to the left of each r_1 and keep going until you place the r_q. When you hit the left boundary of the matrix, you stop for that row.

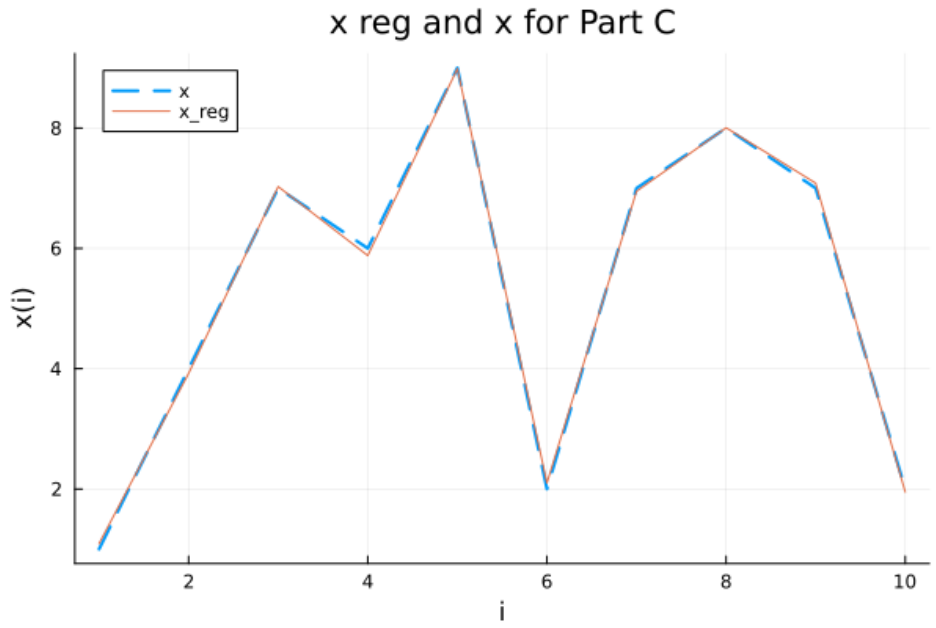
c.

The solution to the regularized least-squares problem is

$$x^{\text{reg}} = (A^\intercal A + \mu I)^{-1} A^\intercal y$$

x^{reg} was plotted alongside x in Julia, the code and plot can be seen below.

```julia
#Question 2.)

#Reading Data + Defining Variables
data2 = readclassjson("recursive.json")
x = data2["x"]
y = data2["y"]
w = data2["w"]

n = 10
k = 5
sigma = 2
q = 10
mu = 0.1
m = k*n

#Define A = BC
C= zeros(m, n)
for col in 1:n
    rows = (k*(col-1)+1):(k*col)
    C[rows, col] .= 1.0
end

r = [exp(-(j^2 / sigma^2)) for j in -q:q]

B = zeros(m, m)
for row in 1:m
    for col in 1:m
        r_idx = row - col
        if abs(r_idx) <= q
            B[row, col] = r[r_idx + (q + 1)]
        end
    end
end

A = B * C

#Part C
x_reg = inv((A' * A) + (mu * I(n))) * A' * y
plot(x, label = "x", lw = 2, linestyle=:dash)
plot!(x_reg, label = "x_reg", lw = 1)
xlabel!("i")
ylabel!("x(i)")
title!("x reg and x for Part C")
savefig("hw5_q2i")
```



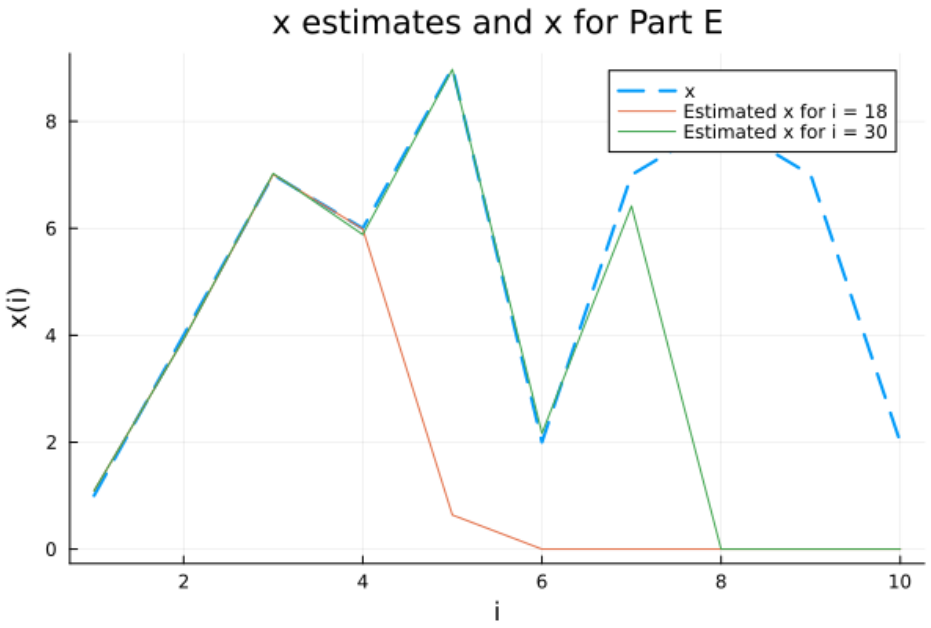
d.

The usual recursive-least-squares algorithm is trying to estimate $x_{ls} = (A^\intercal A)^{-1} A^\intercal y$ where every iteration brings $P(m)$ closer to $A^\intercal A$ and $q(m)$ closer to $A^\intercal y$. We're modifying this algorithm to estimate $x^{\text{reg}} = (A^\intercal A + \mu I)^{-1} A^\intercal y$ instead. The only difference is that $P(m)$ should have a μI term added to it. This has nothing to do with iterating through the rows of A, so the change we make is setting $P(0) = \mu I$ and keeping the rest of the algorithm the same.

e.

This question was done in Julia, the code and plot can be seen below.

```
#Part E
P = mu * I(n)
q = zeros(n)

plot(x, label = "x", lw = 2, linestyle=:dash)
for i in 1:m
    ai = A[i, :]
    yi = y[i]
    global P += ai * ai'
    global q += yi * ai
    xreg = P \ q
    if i == 18
        x_estimate = P \ q
        plot!(x_estimate, label = "Estimated x for i = 18", lw = 1)
    end
    if i == 30
        x_estimate = P \ q
        plot!(x_estimate, label = "Estimated x for i = 30", lw = 1)
    end
end
xlabel!("i")
ylabel!("x(i)")
title!("x estimates and x for Part E")
savefig("hw5_q2iii")
```



f.

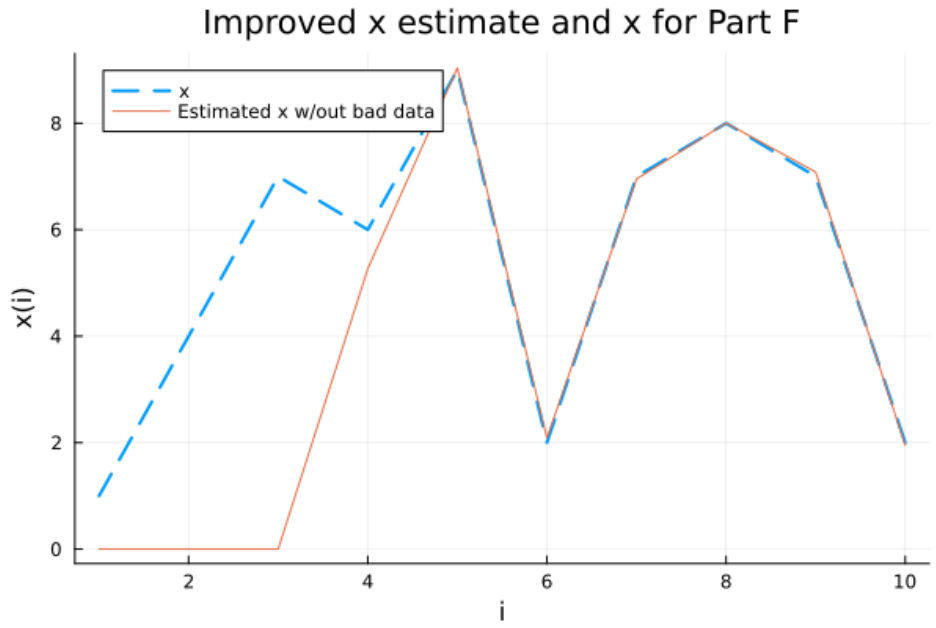
After our recursion in part e, we have

$$P = \mu I + \sum_{i=1}^{50} a_i a_i^\intercal = \mu I + \sum_{i=21}^{50} a_i a_i^\intercal + \sum_{i=1}^{20} a_i a_i^\intercal$$

$$q = \sum_{i=1}^{50} y_i a_i = \sum_{i=21}^{50} y_i a_i + \sum_{i=1}^{20} y_i a_i$$

Our goal is to remove the data from $y_1, ..., y_{20}$ from P and Q. That means we want $P = \mu I + \sum_{i=21}^{50} a_i a_i^\intercal$ and $q = \sum_{i=21}^{50} y_i a_i$. In other words, all we have to do is subtract $\sum_{i=1}^{20} a_i a_i^\intercal$ from P and subtract $\sum_{i=1}^{20} y_i a_i$ from q, then use the modified P and q to compute the new estimate. This was done in Julia, the code and plot of the refined estimate can be seen below.

```
121  #Part F
122  for i in 1:20
123      ai = A[i, :]
124      yi = y[i]
125      global P -= ai * ai'
126      global q -= yi * ai
127  end
128
129  x_improved_est = P \ q
130  plot(x, label = "x", lw = 2, linestyle=:dash)
131  plot!(x_improved_est, label = "Estimated x w/out bad data", lw = 1)
132  xlabel!("i")
133  ylabel!("x(i)")
134  title!("Improved x estimate and x for Part F")
135  savefig("hw5_q2v")
```



8.1110.)

Suppose an explicit reconstruction matrix H exists for the given G and $\tilde{G}$. In this specific case, we know that $H \in \mathbb{R}^{2\times 5}$, so we can represent H as

$$H = \begin{bmatrix} h_{11} & h_{12} & h_{13} & h_{14} & h_{15} \\ h_{21} & h_{22} & h_{23} & h_{24} & h_{25} \end{bmatrix}$$

We then have

$$HG = \begin{bmatrix} h_{11} & h_{12} & h_{13} & h_{14} & h_{15} \\ h_{21} & h_{22} & h_{23} & h_{24} & h_{25} \end{bmatrix} \begin{bmatrix} 2 & 3 \\ 1 & 0 \\ 0 & 4 \\ 1 & 1 \\ -1 & 2 \end{bmatrix} = \begin{bmatrix} (2h_{11} + h_{12} + 0h_{13} + h_{14} - h_{15}) & (3h_{11} + 0h_{12} + 4h_{13} + h_{14} + 2h_{15}) \\ (2h_{21} + h_{22} + 0h_{23} + h_{24} - h_{25}) & (3h_{21} + 0h_{22} + 4h_{23} + h_{24} + 2h_{25}) \end{bmatrix} = I$$

$$\Leftrightarrow \begin{bmatrix} 2 & 1 & 0 & 1 & -1 \\ 3 & 0 & 4 & 1 & 2 \end{bmatrix} \begin{bmatrix} h_{11} \\ h_{12} \\ h_{13} \\ h_{14} \\ h_{15} \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \text{ and } \begin{bmatrix} 2 & 1 & 0 & 1 & -1 \\ 3 & 0 & 4 & 1 & 2 \end{bmatrix} \begin{bmatrix} h_{21} \\ h_{22} \\ h_{23} \\ h_{24} \\ h_{25} \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Similarly

$$H\tilde{G} = \begin{bmatrix} h_{11} & h_{12} & h_{13} & h_{14} & h_{15} \\ h_{21} & h_{22} & h_{23} & h_{24} & h_{25} \end{bmatrix} \begin{bmatrix} -3 & -1 \\ -1 & 0 \\ 2 & -3 \\ -1 & -3 \\ 1 & 2 \end{bmatrix}$$

$$= \begin{bmatrix} (-3h_{11} - h_{12} + 2h_{13} - h_{14} + h_{15}) & (-h_{11} + 0h_{12} + -3h_{13} - 3h_{14} + 2h_{15}) \\ (-3h_{21} - h_{22} + 2h_{23} - h_{24} + h_{25}) & (-h_{21} + 0h_{22} + -3h_{23} - 3h_{24} + 2h_{25}) \end{bmatrix} = I$$

$$\Leftrightarrow \begin{bmatrix} -3 & -1 & 2 & -1 & 1 \\ -1 & 0 & -3 & -3 & 2 \end{bmatrix} \begin{bmatrix} h_{11} \\ h_{12} \\ h_{13} \\ h_{14} \\ h_{15} \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \text{ and } \begin{bmatrix} -3 & -1 & 2 & -1 & 1 \\ -1 & 0 & -3 & -3 & 2 \end{bmatrix} \begin{bmatrix} h_{21} \\ h_{22} \\ h_{23} \\ h_{24} \\ h_{25} \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Therefore if we can solve the following equations for all h_{ij} then we're done

$$\begin{bmatrix} 2 & 1 & 0 & 1 & -1 \\ 3 & 0 & 4 & 1 & 2 \\ -3 & -1 & 2 & -1 & 1 \\ -1 & 0 & -3 & -3 & 2 \end{bmatrix} \begin{bmatrix} h_{11} \\ h_{12} \\ h_{13} \\ h_{14} \\ h_{15} \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

$$\begin{bmatrix} 2 & 1 & 0 & 1 & -1 \\ 3 & 0 & 4 & 1 & 2 \\ -3 & -1 & 2 & -1 & 1 \\ -1 & 0 & -3 & -3 & 2 \end{bmatrix} \begin{bmatrix} h_{21} \\ h_{22} \\ h_{23} \\ h_{24} \\ h_{25} \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 1 \end{bmatrix}$$

We can determine if these are solvable, and if they are solve them, by finding the augmented matrices and converting them to reduced row echelon form:

$$\text{rref}(\begin{bmatrix} 2 & 1 & 0 & 1 & -1 & 1 \\ 3 & 0 & 4 & 1 & 2 & 0 \\ -3 & -1 & 2 & -1 & 1 & 1 \\ -1 & 0 & -3 & -3 & 2 & 0 \end{bmatrix}) = \begin{bmatrix} 1 & 0 & 0 & 0 & 0.64 & -0.72 \\ 0 & 1 & 0 & 0 & -1.08 & 2.84 \\ 0 & 0 & 1 & 0 & 0.32 & 0.64 \\ 0 & 0 & 0 & 1 & -1.2 & -0.4 \end{bmatrix} \Rightarrow \begin{bmatrix} h_{11} \\ h_{12} \\ h_{13} \\ h_{14} \\ h_{15} \end{bmatrix} = \begin{bmatrix} -0.72 \\ 2.84 \\ 0.64 \\ -0.4 \\ 0 \end{bmatrix} \text{ is a solution}$$

$$\text{rref}(\begin{bmatrix} 2 & 1 & 0 & 1 & -1 & 0 \\ 3 & 0 & 4 & 1 & 2 & 1 \\ -3 & -1 & 2 & -1 & 1 & 0 \\ -1 & 0 & -3 & -3 & 2 & 1 \end{bmatrix}) = \begin{bmatrix} 1 & 0 & 0 & 0 & 0.64 & 0.32 \\ 0 & 1 & 0 & 0 & -1.08 & -0.04 \\ 0 & 0 & 1 & 0 & 0.32 & 0.16 \\ 0 & 0 & 0 & 1 & -1.2 & -0.6 \end{bmatrix} \Rightarrow \begin{bmatrix} h_{21} \\ h_{22} \\ h_{23} \\ h_{24} \\ h_{25} \end{bmatrix} = \begin{bmatrix} 0.32 \\ -0.04 \\ 0.16 \\ -0.6 \\ 0 \end{bmatrix} \text{ is a solution}$$

Hence, an explicit reconstruction matrix H for given G and $\tilde{G}$ is

$$H = \begin{bmatrix} -0.72 & 2.84 & 0.64 & -0.4 & 0 \\ 0.32 & -0.04 & 0.16 & -0.6 & 0 \end{bmatrix}$$

This was verified in Julia, as can be seen below.

```
#Question 3.)
G = [2 3;
     1 0;
     0 4;
     1 1;
     -1 2]

G_tilde = [-3 -1;
           -1  0;
            2 -3;
           -1 -3;
            1  2]

H = [-0.72  2.84 0.64 -0.4 0;
      0.32 -0.04 0.16 -0.6 0]
println()
println("HG = ", H*G)
println("HG_tilde = ", H*G_tilde)
println()
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL
TERMINAL

HG = [0.9999999999999999 -1.1102230246251565e-16; 0.0 1.0]
HG_tilde = [1.0000000000000004 -1.1102230246251565e-16; 1.1102230246251565e-16 0.9999999999999999]
```

8.1150.)

a.

The conservation equations can be represented by $Af + s = 0$ and the mean square traffic can be represented as $\frac{\|f\|^2}{n}$. This problem is then equivalent to finding the least norm solution to $Af = -s$ which is

$$f_{ln} = A^\intercal(AA^\intercal)^{-1}(-s) = -A^\intercal(AA^\intercal)^{-1}s$$

*Note for this solution to hold, A must be full rank and fat, which is true if there are more links than nodes and the rows of A are independent of each other.

b.

This question was done in Julia, the code and results can be seen below. As can be seen, the mean square traffic for the optimal flow is 54.37 and the mean square traffic for the simple flow is 77.29. This means the optimal flow reduces the mean square traffic compared to the simple flow, which is to be expected.

```
161  #Question 4.)
162
163  A = [-1 1 -1 -1 0 0 0;
164       0 -1 0 0 -1 0 0;
165       0 0 0 1 1 -1 0;
166       0 0 1 0 0 1 -1]
167  s = [1 4 10 10]'
168  n = 7
169  f_simple = [5 4 0 0 0 10 20]'
170
171  f = -A' * inv(A * A') * s
172
173  println("f_optimal Mean Square Error = ", norm(f)^2 / n)
174  println("f_simple Mean Square Error Ecom = ", norm(f_simple)^2 / n)
175
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL
> TERMINAL

f_optimal Mean Square Error = 54.37414965986394
f_simple Mean Square Error Ecom = 77.2857142857143
```

8.1230.)

a.

Since C is not full rank that means the rows of C are linearly dependent. As $K = \begin{bmatrix} A^\intercal A & C^\intercal \\ C & 0 \end{bmatrix}$, this implies that the bottom k rows (the rows for $\begin{bmatrix} C & 0 \end{bmatrix}$) are linearly dependent as well. This means that the rows of K are linearly dependent and since K is a square matrix, we have that K isn't full rank and thus singular.

b.

As K is a square matrix, to prove that K is singular it's sufficient to show that $\text{null}(K) \neq \{0\}$. Consider $\begin{bmatrix} u \\ 0 \end{bmatrix}$ where $u \in \mathbb{R}^n$ as the problem described and $0 \in \mathbb{R}^k$. Then

$$K\begin{bmatrix} u \\ 0 \end{bmatrix} = \begin{bmatrix} A^\intercal A & C^\intercal \\ C & 0 \end{bmatrix}\begin{bmatrix} u \\ 0 \end{bmatrix} = \begin{bmatrix} A^\intercal(Au) \\ Cu \end{bmatrix} = 0 \Rightarrow 0 \neq \begin{bmatrix} u \\ 0 \end{bmatrix} \in \text{null}(K)$$

Therefore, the null space of K is nonzero, so K is singular.

c.

We want to use the given argument to conclude that either C is not full rank or $\text{null}(A) \cap \text{null}(C) \neq \{0\}$. Assume that $\text{null}(A) \cap \text{null}(C) = \{0\}$. If we can prove that C must be full rank, we're done. As $u \in \text{null(A)}$ and $u \in \text{null}(C)$ we have that $u \in \text{null}(A) \cap \text{null}(C)$, so $u = 0$. We know that $\begin{bmatrix} u \\ v \end{bmatrix}$ is a nonzero vector, which means that v must be nonzero. Therefore,

$$\begin{bmatrix} A^\intercal A & C^\intercal \\ C & 0 \end{bmatrix} \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} A^\intercal A & C^\intercal \\ C & 0 \end{bmatrix} \begin{bmatrix} 0 \\ v \end{bmatrix} = 0 \Rightarrow \begin{bmatrix} C^\intercal v \\ 0 \end{bmatrix} = 0 \Rightarrow 0 \neq v \in \text{null}(C^\intercal)$$

This means the null space of $C^\intercal$ is nonzero, and since $C^\intercal$ is skinny, we have that $C^\intercal$ isn't full rank which implies that C isn't full rank. Hence, either $\text{null}(A) \cap \text{null}(C) \neq \{0\}$ or C is not full rank.